



南开大学
Nankai University

《软件测试与维护》实验报告

(2025~2026 学年第二学期)

实验名称：黑盒测试实验

学 院：软件学院

姓 名：李宗杭

学 号：2311451

指导老师：张圣林

2026 年 6 月 4 日

目录

1 实验背景	1
2 实验环境	1
3 实验步骤	2
3.1 Selenium IDE 功能自动化测试	2
3.2 JMeter 性能测试	6
4 实验结果	8
4.1 Selenium 功能测试结果	8
4.2 JMeter 性能测试结果	9
5 实验分析	9
5.1 Selenium 测试问题分析	9
5.2 JMeter 性能分析	10
6 实验总结与心得体会	10
6.1 遇到的问题及解决方法	10
6.2 心得感悟	10

实验 3：黑盒测试实验

1 实验背景

黑盒测试是一种重要的软件测试方法，它将软件系统视为一个无法打开的黑盒子，完全不考虑系统内部的逻辑结构和实现细节，仅通过验证系统的输入与输出是否符合需求规格说明书来评估软件质量。黑盒测试主要关注软件的功能完整性、正确性和易用性，是软件测试流程中不可或缺的一环。

本次实验将使用两款主流的黑盒测试工具：Selenium IDE 和 Apache JMeter。Selenium IDE 是一款开源的 Web 自动化测试工具，通过录制用户在浏览器中的操作来生成测试脚本，能够快速实现 Web 应用的功能自动化测试。Apache JMeter 则是一款功能强大的性能测试工具，支持模拟大量并发用户访问，能够测试系统在不同负载下的响应时间、吞吐量、错误率等关键性能指标，帮助识别系统的性能瓶颈。

通过本次实验，旨在掌握 Selenium IDE 的录制、调试和脚本导出方法，以及 JMeter 的性能测试流程和报告分析技巧，深入理解黑盒测试的基本原理和实际应用。

2 实验环境

- 操作系统：Windows 11 家庭版
- 浏览器：Microsoft Edge 148.0.3967.96
- Java 版本：OpenJDK 21.0.6
- Selenium IDE 版本：3.17.0
- Apache JMeter 版本：5.6.3
- Python 版本：3.12.11
- pytest 版本：9.0.3

3 实验步骤

3.1 Selenium IDE 功能自动化测试

安装 Selenium IDE

打开 Microsoft Edge 浏览器，进入 Edge 扩展商店，搜索”Selenium IDE”，点击”获取”按钮完成安装。安装完成后，浏览器右上角扩展栏中会出现 Selenium IDE 图标。

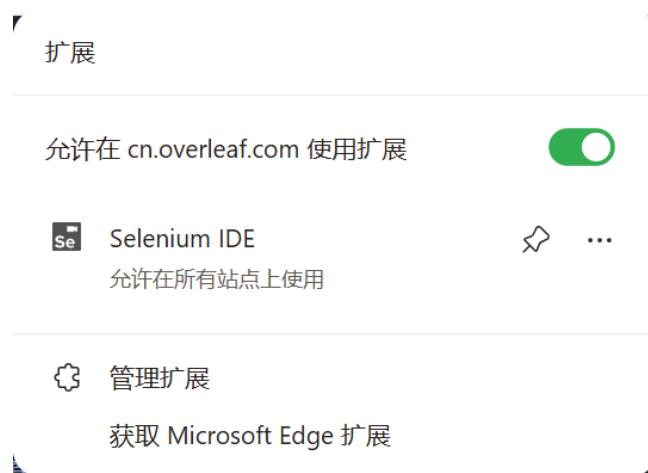


图 1: Selenium IDE 安装截图

创建测试项目

点击浏览器右上角的 Selenium IDE 图标，选择”Create a new project”，输入项目名称”DraggableTest”，点击”OK”创建新项目。

录制 Simple list 测试用例

1. 在地址栏中输入测试网址：<https://shopify.github.io/draggable/examples/>
2. 点击录制按钮开始录制
3. 在浏览器中点击左侧导航栏的”Simple list”链接
4. 拖拽列表中的第一个元素”item one”到第三个元素”item three”的位置
5. 拖拽列表中的第二个元素”item two”到第四个元素”item four”的位置
6. 点击停止录制按钮，输入测试用例名称”test_simplelist”

调试测试用例

初始录制的测试用例使用坐标定位元素，执行时出现失败，错误信息为”Implicit Wait timed out after 30000ms”。

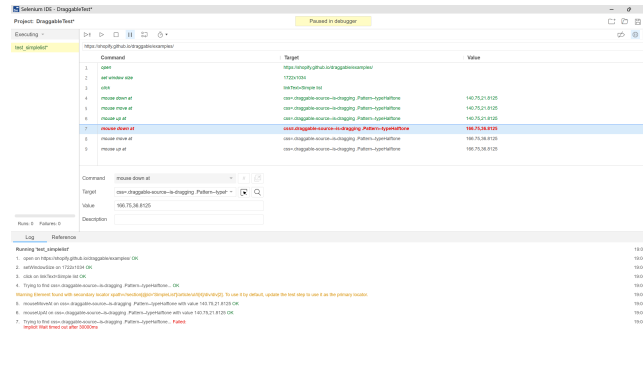


图 2: Simple list 失败截图

向 AI 寻求建议，尝试使用 XPath 定位器，虽然测试执行显示成功，但浏览器页面元素并未发生实际变化，于是自行探索解决方案。通过浏览器 F12 开发者工具查看元素属性，发现每个列表项都有唯一的类名”StackedListItem-itemX”。

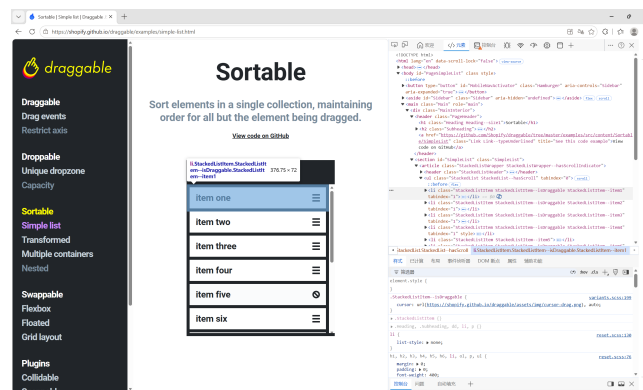


图 3: 浏览器 F12 查看元素类名截图

将测试命令从”mouse down at”、”mouse move at”、”mouse up at” 修改为”drag and drop to object”，并使用类名作为定位器：”css=.StackedListItem-item1”。

修改后重新执行测试用例，测试成功通过，且浏览器页面元素正确完成拖拽。

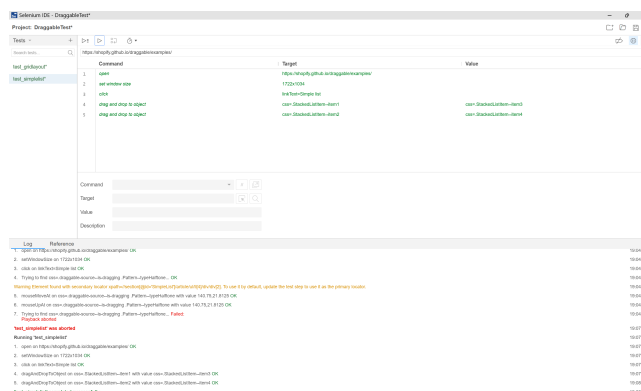


图 4: 修正后 Simple list 测试成功截图

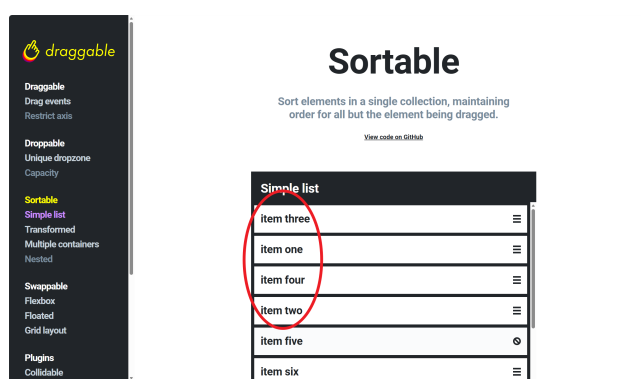


图 5: Simple list 拖拽后页面效果

录制 Grid layout 测试用例

按照相同的步骤录制 Grid layout 功能的测试用例：

1. 新建测试用例”test_gridlayout”
2. 点击录制按钮，访问测试网址
3. 点击左侧导航栏的”Grid layout” 链接
4. 拖拽第一个元素到第二个元素的位置
5. 拖拽第二个元素到第四个元素的位置
6. 停止录制，使用相同的方法修改元素定位方式

初始测试同样出现坐标定位不稳定的问题，修改为类名定位后测试成功执行。

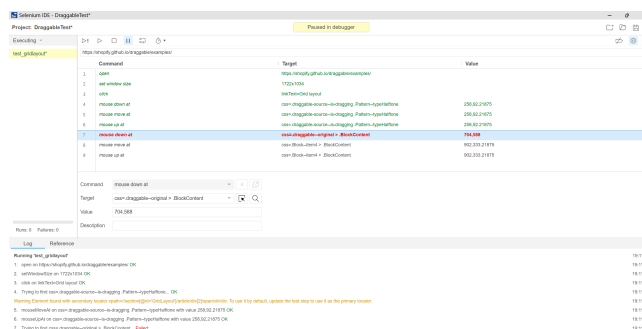


图 6: Grid layout 失败截图

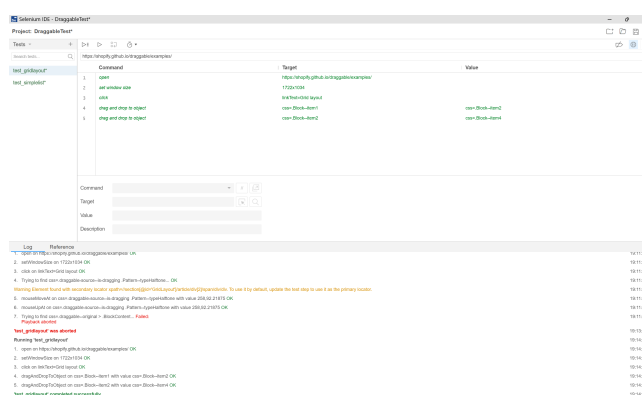


图 7: 修正后 Grid layout 测试成功截图

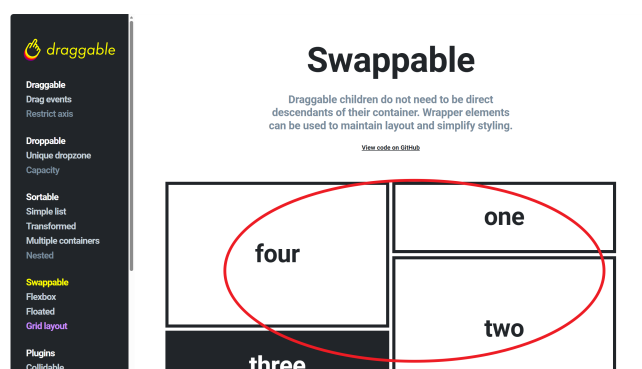


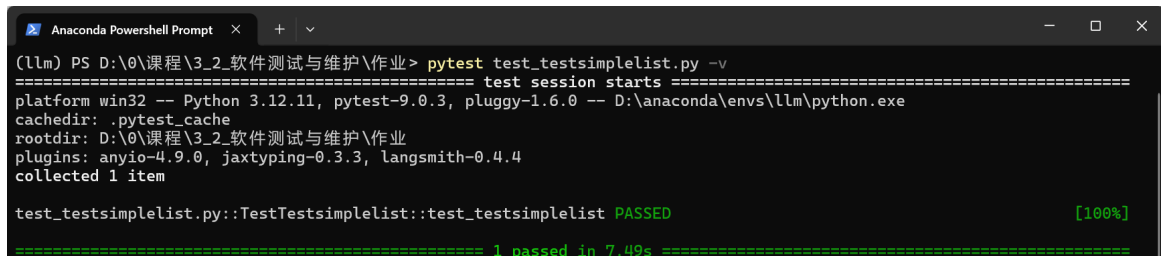
图 8: Grid layout 拖拽后页面效果

导出并运行 Python 脚本

在 Selenium IDE 中, 右键点击测试用例“test_simplelist”, 选择“Export”, 导出为“Python pytest” 格式的脚本。在本地终端中执行以下命令运行脚本:

```
1 pytest test_testsimplelist.py -v
```

脚本成功执行，验证了测试用例的正确性。



```
Anaconda PowerShell Prompt
(11m) PS D:\0\课程\3_2_软件测试与维护\作业> pytest test_testsimplelist.py -v
===== test session starts =====
platform win32 -- Python 3.12.11, pytest-9.0.3, pluggy-1.6.0 -- D:\anaconda\envs\11m\python.exe
cachedir: .pytest_cache
rootdir: D:\0\课程\3_2_软件测试与维护\作业
plugins: anyio-4.9.0, jaxtyping-0.3.3, langsmith-0.4.4
collected 1 item

test_testsimplelist.py::TestTestsimplelist::test_testsimplelist PASSED [100%]

===== 1 passed in 7.49s =====
```

图 9: Python 脚本运行结果截图

3.2 JMeter 性能测试

安装配置 JMeter

1. 安装 Java 8 及以上版本，配置 JAVA_HOME 环境变量
2. 从 Apache JMeter 官网下载最新版本压缩包
3. 解压压缩包到本地目录，将 bin 目录添加到系统 PATH 环境变量
4. 在命令行中输入”jmeter -v” 验证安装成功

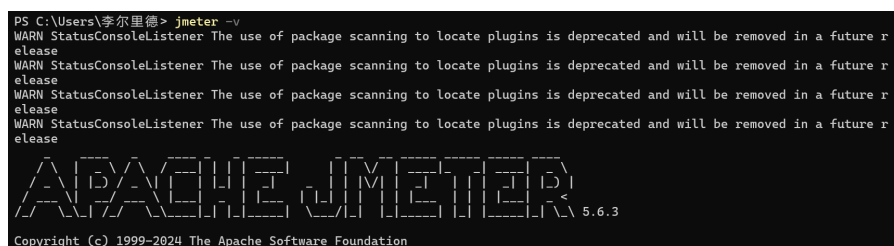


图 10: JMeter 版本验证截图

导入测试计划

从仓库”<https://github.com/harsha123e/JMeter-BlazeDemo-Test>” 下载测试计划，打开 JMeter GUI，点击”File” -> ”Open”，选择下载的”BlazeDemo_Test_Plan.jmx” 文件，导入测试计划。该测试计划包含了 BlazeDemo 航班预订系统的完整用户流程：首页访问、航班选择、航班预订、确认页面。

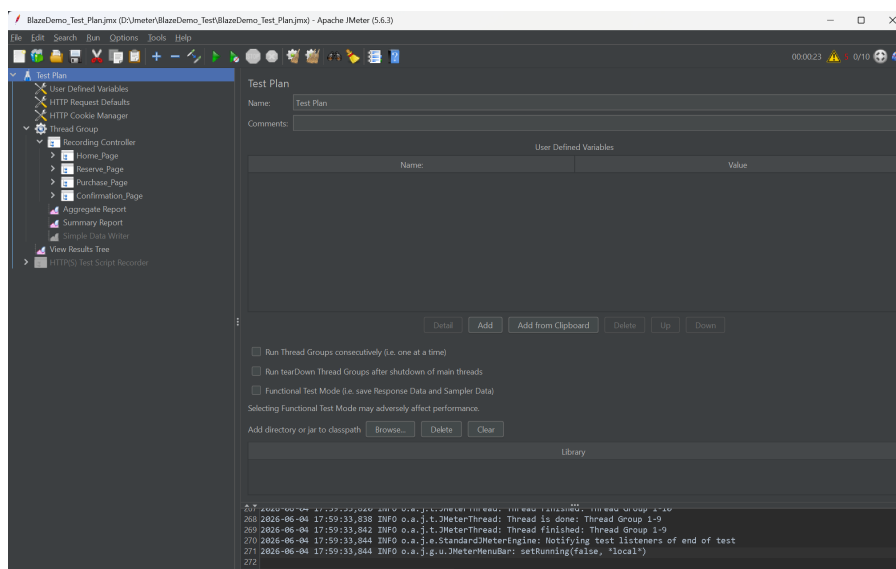


图 11: JMeter 测试计划导入

运行负载测试

使用 CLI 模式运行测试。在命令行中执行以下命令：

```
1 jmeter -n -t BlazeDemo_Test_Plan.jmx -l results/results.csv -e -o ...  
    results/Reports
```

命令说明：

- -n: 使用非 GUI 模式运行
- -t: 指定测试计划文件
- -l: 指定结果文件保存路径
- -e: 生成 HTML 报告
- -o: 指定 HTML 报告输出目录

分析性能报告

测试完成后，打开 results/Reports 目录下的 index.html 文件，查看详细的性能报告。

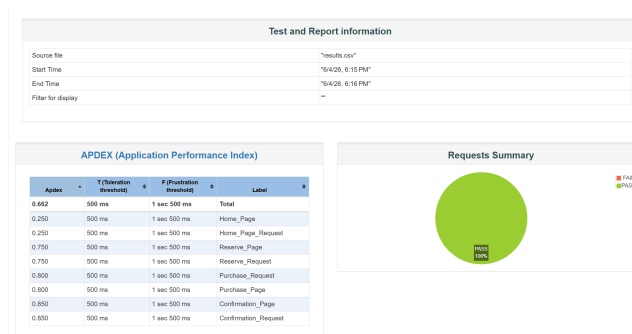


图 12: JMeter 测试报告-1

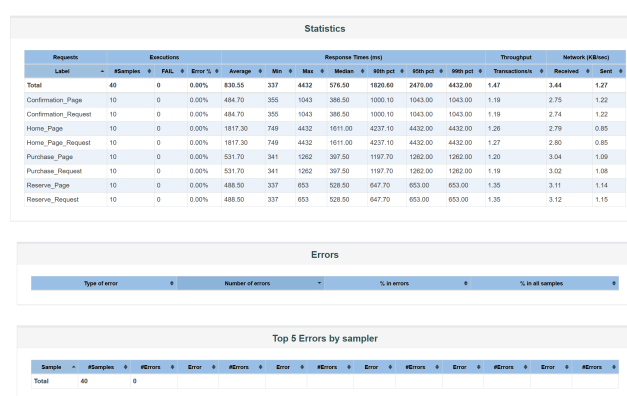


图 13: JMeter 测试报告-2

4 实验结果

4.1 Selenium 功能测试结果

本次实验成功完成了 Draggable 网站 Simple list 和 Grid layout 两个功能的自动化测试：

- 解决了初始录制中坐标定位不稳定的问题，通过使用类名定位器实现了稳定的拖拽操作
- 两个测试用例在 Selenium IDE 中均成功执行，页面元素正确完成拖拽
- 导出的 Python pytest 脚本在本地环境中成功运行，验证了测试用例的可移植性

4.2 JMeter 性能测试结果

本次 JMeter 性能测试模拟了 10 个并发用户访问 BlazeDemo 航班预订系统，测试结果如下：

- 总请求数：40 次
- 错误率：0.00
- 平均响应时间：830.55ms
- 最小响应时间：337ms
- 最大响应时间：4432ms
- 吞吐量：1.47 transactions/sec
- 整体 APDEX 指数：0.662，系统性能一般

页面名称	平均响应时间 (ms)	90% 响应时间 (ms)	APDEX 指数
Home_Page	1817.30	4237.10	0.250
Reserve_Page	488.50	647.70	0.750
Purchase_Page	531.70	1197.70	0.800
Confirmation_Page	484.70	1000.10	0.850

表 1: 各页面详细性能指标

5 实验分析

5.1 Selenium 测试问题分析

本次 Selenium 测试中遇到的主要问题是元素定位不稳定：

1. 坐标定位的局限性：初始录制使用的坐标定位方式受浏览器窗口大小、屏幕分辨率影响极大，一旦窗口大小发生变化，测试就会失败。
2. XPath 定位的健壮性问题：XPath 定位虽然能够找到元素，但在拖拽操作中不够稳定，可能导致操作不生效。
3. 类名定位的优势：通过 F12 开发者工具找到的唯一类名定位器，具有稳定性高、可读性好的优点，非常适合拖拽这类需要精确元素定位的操作。

5.2 JMeter 性能分析

从 JMeter 测试报告可以看出, BlazeDemo 系统存在明显的性能瓶颈:

1. 首页性能严重不足: 首页平均响应时间高达 1817ms, 90% 的用户需要等待超过 4 秒才能加载完成, APDEX 指数仅为 0.25, 远低于良好标准 (0.75 以上)。这可能是由于首页包含大量静态资源未进行优化, 或者服务器端处理逻辑效率低下。
2. 其他页面性能良好: 预订流程中的其他页面响应时间均在 500ms 左右, APDEX 指数在 0.75-0.85 之间, 性能表现良好。
3. 系统稳定性良好: 在 10 个并发用户的负载下, 系统没有出现任何错误, 说明系统的稳定性能够满足基本需求。

针对以上问题, 提出以下优化建议:

- 对首页的静态资源 (图片、CSS、JavaScript) 进行压缩和合并, 使用 CDN 加速
- 优化首页的服务器端处理逻辑, 减少数据库查询次数
- 实现首页的懒加载, 只加载用户可见区域的内容
- 增加服务器的带宽和计算资源, 提高系统的并发处理能力

6 实验总结与心得体会

6.1 遇到的问题及解决方法

1. Selenium IDE 录制的测试用例执行失败: 初始使用坐标定位不稳定, 通过浏览器 F12 开发者工具查看元素属性, 改用类名定位器和 dragAndDropToObject 命令解决。
2. Python 脚本运行缺少依赖: 导出的 Python 脚本需要安装 selenium 和 pytest 库, 通过 pip install 命令安装所需依赖后成功运行。

6.2 心得感悟

通过本次黑盒测试实验, 我深入理解了黑盒测试的基本原理和实际应用, 掌握了 Selenium IDE 和 JMeter 两款主流测试工具的使用方法。

在 Selenium IDE 的使用过程中，我认识到元素定位是自动化测试的核心，选择合适的定位器直接影响测试用例的稳定性和可维护性。坐标定位虽然简单，但极其脆弱；类名和 ID 定位则更加稳定可靠，是自动化测试的首选。

在 JMeter 性能测试中，我学会了如何模拟并发用户访问，如何生成和分析性能报告。通过对测试结果的分析，我能够识别系统的性能瓶颈，并提出针对性的优化建议。这让我认识到性能测试不仅是发现问题，更重要的是解决问题，提升系统的用户体验。

本次实验也让我体会到了软件测试的重要性。一个高质量的软件产品不仅需要功能正确，还需要有良好的性能表现，我们不仅要学会编写代码，还要学会测试代码，确保我们开发的软件能够稳定、高效地运行。